

BatchedSRNNCell — Full Equations, Parameters, Transforms

Faithful description of every equation, parameter, transform, init, and ablation mask in `train_s rnn/models/s rnn_cell.py::BatchedSRNNCell`. All line numbers in this document refer to that file.

The cell runs K SRNN ablation variants in parallel via `torch.bmm`. A leading K axis appears on every parameter and state tensor; the single-cell `SRNNCell` (lines 172–799) drops that axis but otherwise performs the same math (with two minor differences flagged in `KnownIssues.md` §1).

0. Reading guide & notation

- K — number of stacked ablation variants (`self.K`).
- B — batch size, T — sequence length, N — `num_units`.
- $n_E = \lfloor N/2 \rfloor$, $n_I = N - n_E$ — E / I partition sizes.
- D — `input_size`, O — readout output size.
- h — wall-clock step (`cfg.h`, default 0.02 s).
- u_f — `cfg.ode_unfolds` (default 1).
- $\Delta t = h/u_f$ — solver substep used inside one `forward` call (line 1722).
- $n_{aE}^{\max}$, $n_{aI}^{\max}$ — max SFA tier counts across the K configs (`self.max_n_a_E`, `self.max_n_a_I`); inactive tiers are zero-padded then masked.
- $n_{bE}^{\max}$, $n_{bI}^{\max} \in \{0, 1\}$ analogously for STD.

Each equation block below is given **twice**:

1. **Code form** — using the literal Python attribute names so you can grep the source.
2. **LaTeX form** — using single-letter symbols suitable for a paper.

A symbol cheatsheet is at the end (§ 13).

1. Transform reference

Two parameter-positivity transforms are used:

transform	code	$f(\theta)$	inverse used at init	init helper (lines 26–37)
softplus	<code>F.softplus</code>	$\log(1 + e^\theta)$	$f^{-1}(y) = \log(e^y - 1)$	<code>inv_softplus(y)</code> (scalar), <code>_softplus_inv(y)</code> (tensor)
exp	<code>torch.exp</code>	e^θ	$f^{-1}(y) = \log y$	plain <code>math.log</code> (init at $\log 1 = 0$)

Naming convention. Parameters are prefixed by the transform that makes them positive at use-time:

- `isp_...` — stored in **inverse-softplus space**, transformed via `F.softplus` at use-time. (`isp` for “inverse softplus.”)
- `log_..._gain` — stored in **true log space**, transformed via `torch.exp` at use-time. Exclusively the per-variant scalar gains.

Identifying the transform is now a pure prefix lookup. (Earlier the inverse-softplus params were also prefixed `log_...`, which collided visually with the genuine log-space gains; that was renamed in this doc’s companion refactor.)

Concretely, for any time constant $\tau \in \{\tau_d, \tau_a^E, \tau_a^I, \tau_b^{\text{rec}, E}, \tau_b^{\text{rel}, E}, \tau_b^{\text{rec}, I}, \tau_b^{\text{rel}, I}\}$:

$$\tau = \tau_{\text{global}} \cdot \underbrace{\exp(\theta_{\text{gain}})}_{\text{exp on log-gain}} \cdot \underbrace{\text{softplus}(\theta_{\text{vec}})}_{\text{softplus on inverse-softplus init}}$$

i.e. each τ is built from **three** factors with **two different positivity transforms** stacked. They are mathematically equivalent in the sense that both maps $\mathbb{R} \rightarrow \mathbb{R}_{>0}$, but they have different gradient curvatures (softplus saturates linearly for large positive

inputs; exp grows multiplicatively). This asymmetry is what looked sloppy at the documentation pass and is explicitly called out here.

Identity (no positivity transform). A handful of parameters are stored in their natural space and can take any sign:

param	shape	init	semantics
W_raw_gain	$(K,)$	1.0	recurrent gain (ESN spectral-radius knob); free real
W_in_gain	$(K,)$	1.0	input gain; free real
W_out_gain	$(K,)$	1.0	readout-weight gain (lives on <code>SequenceModel</code> , not the cell); free real; bias unaffected
a_0_scalar	$(K,)$	0.0	shared additive threshold offset
c_0_E_scalar, c_0_I_scalar	$(K,)$	0.0	shared additive SFA-offset
c_0_E_vec, c_0_I_vec	$(K, n_*, n_a^{\max})$	0.0	per-neuron SFA offsets
a_0_vec	(K, N)	0.35	per-neuron threshold

Of these, `W_raw_gain`, `W_in_gain`, and `W_out_gain` initialise to +1 (multiplicative identity) but are *not* in log space — Adam can drive them negative, which would invert the sign of the corresponding linear projection. This is intentional in the current design (it lets the optimizer flip overall gain polarity if needed) but is a candidate for the cleanup.

The threshold split. The effective threshold is the sum of a vec and a scalar:

`a_0 = a_0_vec + a_0_scalar` # $(K, N) + (K,) \rightarrow (K, N)$

When `per_neuron=False`, `a_0_vec`'s gradient is zeroed by a hook (line 1034) so only `a_0_scalar` moves and the per-neuron values stay locked at their identical init (0.35). When `per_neuron=True`, both train: the scalar captures the shared direction and the vec captures per-neuron deviations.

The same scalar+vec split applies to every per-neuron SFA / STD / dendritic parameter (`isp_tau_d`, `isp_tau_a_E`, `isp_c_E`, `c_0_E`, `isp_tau_b_rec_E`, `isp_tau_b_rel_E`, and the I-side mirrors). The vec component is gradient-masked off in `per_neuron=False` variants.

2. Configuration & ablation switches

Each of the K stacked variants is described by an `SRNNConfig` dataclass (lines 82–119). The fields and their defaults:

field	default	meaning
num_units	32	N
dales	True	apply Dale's softplus + sign
n_a_E, n_a_I	3, 3	SFA tier counts (0 disables)
n_b_E, n_b_I	1, 1	STD presence (0 disables, 1 enables)
per_neuron	False	unmask per-neuron vec gradients
echo	False	freeze <code>W_raw</code> (reservoir mode)
skip	False	post-readout residual $y += x_{in}$ (autoregressive only)
solver	"semi_implicit"	one of <code>semi_implicit</code> , <code>explicit</code> , <code>rk4</code> , <code>exponential</code>
h	0.02	wall-clock step (s)
ode_unfolds	1	sub-stepping inside <code>forward</code>
readout	"synaptic"	one of <code>synaptic</code> , <code>rate</code> , <code>dendritic</code>
tau_global_init	1.0	initial value of τ_{global}
std_zero_floor	True	rescale $b \rightarrow (b - b_{\min}) / (1 - b_{\min})$ at readout

`solver`, `h`, `ode_unfolds` must be identical across the K stacked configs (asserted at lines 1101–1111). All other fields can vary across K and are converted into multiplicative masks (§ 9).

A registry of preset configs lives in `SRNN_PRESETS` (lines 126–165); new ablations are typically added by editing that dict and the matching YAML in `conf/model/srnn_*.yaml`.

3. State layout & initialization

`init_state` (lines 1304–1320) returns a packed flat tensor of shape $(K, B, S_{\max})$ where

$$S_{\max} = n_E n_{aE}^{\max} + n_I n_{aI}^{\max} + n_E n_{bE}^{\max} + n_I n_{bI}^{\max} + N$$

with packing order $[a_E | a_I | b_E | b_I | x]$ (`unpack_state` at lines 1236–1279, `pack_state` at lines 1281–1293).

Initial values (hard-coded, matches MATLAB `SRNNModel2.initialize_state`):

state	shape	init
<code>a_E</code>	$(K, B, n_E, n_{aE}^{\max})$	0
<code>a_I</code>	$(K, B, n_I, n_{aI}^{\max})$	0
<code>b_E</code>	(K, B, n_E)	1 (synapses fully available)
<code>b_I</code>	(K, B, n_I)	1
<code>x</code>	(K, B, N)	$0.1 \cdot \mathcal{N}(0, 1)$

Inactive tiers (e.g. `n_a_E=0`) reserve no slots in the packed tensor; `unpack_state` returns synthetic zero / one tensors of the right shape so downstream code can branch-freely (lines 1252, 1260, 1267, 1274).

4. Parameters — code-form effective expressions

Below, each effective quantity appears once with its **code-name formula** (literal attribute names), its **defining line(s)**, the **stored shape**, the **initialization**, and notes on overrides / gradient masks. The companion LaTeX symbol is in column 1 to make the table searchable.

4.1 Recurrent weight W^{eff} — line 1117

Code form (line 1129–1131):

```
W_eff = W_raw_gain * ( dales_mask * dales_signs * softplus(W_raw)
                      + (1 - dales_mask) * W_raw ) * sparsity_masks
```

component	shape	init	source / override
<code>W_raw</code>	(K, N, N)	from <code>rmt_export["W_init"]</code>	RMTMatrix builder (factory.py); not user-tweakable per-config
<code>W_raw_gain</code>	$(K,)$	1.0 (line 877)	always trainable, hard-coded init
<code>sparsity_masks</code>	(K, N, N) buffer	from <code>rmt_export["sparsity_mask"]</code>	RMTMatrix
<code>dales_signs</code>	(K, N) buffer	from <code>rmt_export["dales_sign"]</code>	RMTMatrix;
<code>dales_mask</code>	$(K, 1, 1)$ buffer	<code>float(c.dales)</code> per-variant (line 991)	column-broadcast from <code>cfg.dales</code>
<code>echo_flags</code>	$(K, 1, 1)$ buffer	<code>float(c.echo)</code> (line 995)	from <code>cfg.echo</code> ; gradient hook on <code>W_raw</code> (line 1008) zeros gradient for echo variants — <code>W_raw_gain</code> still trains

Echo gradient masking (lines 1004–1008):

```
echo_grad_mask = 1.0 - echo_flags          # (K, 1, 1): 0 for echo, 1 for non-echo
self.W_raw.register_hook(lambda grad: grad * self._echo_grad_mask)
```

Echo variants therefore have a frozen recurrent **structure** but a trainable per-variant scalar **gain** (`W_raw_gain`).

4.2 Input weight $W^{\text{in,eff}}$ — line 1423

Code form (line 1440–1443):

```
W_in_eff = W_in_gain * (W_in * W_in_mask)    # if W_in_mask provided
          = W_in_gain * W_in                  # otherwise
```

component	shape	init	source / override
<code>W_in</code>	(K, N, D)	$0.1 \cdot \mathcal{N}(0, 1)$ (line 864)	hard-coded scale
<code>W_in_gain</code>	$(K,)$	1.0 (line 878)	always trainable
<code>W_in_mask</code>	$(1, N, 1)$ buffer	from external <code>W_in_mask</code> arg	typically the input-neuron partition (~25 % of units); built by <code>factory.build_batched_model</code>

The drive is then $u = W^{\text{in,eff}} \cdot u_{\text{ext}}$, computed batched as `u = inputs @ Win,eff` (line 1447).

4.3 Threshold a_0 — line 1212

Code form:

```
a_0 = a_0_vec + a_0_scalar          # (K, N) + (K,1) broadcast
```

component	shape	init (hard-coded)	gradient mask
<code>a_0_vec</code>	(K, N)	0.35 (line 885)	zeroed when <code>per_neuron=False</code> (line 1034)
<code>a_0_scalar</code>	$(K,)$	0.0 (line 886)	always trains

4.4 Global timescale τ_{global} — line 1135

Code form:

```
tau_global = softplus(isp_tau_global)      # (K,)
```

component	shape	init	override?
<code>isp_tau_global</code>	$(K,)$	<code>inv_softplus(c.tau_global_init)</code> default <code>inv_softplus(1.0)</code> (lines 889–891)	variant via <code>SRNNConfig.tau_global_init</code>

4.5 Dendritic time constant τ_d — line 1139

Code form (3-factor decomposition):

```
tau_d = tau_global.unsqueeze(-1)          # (K, 1)
        * exp(log_tau_d_gain).view(K, 1)  # (K, 1)
        * softplus(isp_tau_d_vec)         # (K, N)
```

component	shape	init	gradient mask
<code>isp_tau_d_vec</code>	(K, N)	<code>inv_softplus(0.1)</code> (line 896) $\rightarrow$ effective $\tau_d \approx 0.1$ s base	zeroed when <code>per_neuron=False</code> (line 1036)

component	shape	init	gradient mask
log_tau_d_gain	(K,)	0.0 $\rightarrow$ exp = 1 (line 897)	always trains

Note the asymmetry: `isp_tau_d_vec` uses **softplus**, `log_tau_d_gain` uses **exp**. Same pattern repeats for every other τ below.

4.6 SFA time constants τ_a^E, τ_a^I — lines 1144, 1149

Code form (E side; I side mirrors with $n_E \rightarrow n_I$):

```
tau_a_E = tau_global.reshape(K, 1, 1)
    * exp(log_tau_a_E_gain).view(K, 1, 1)      # (K, 1, 1)
    * softplus(isp_tau_a_E_vec)                  # (K, n_E, max_n_a_E)
```

component	shape	init	gradient mask
isp_tau_a_E_vec	(K, $n_E, n_{aE}^{\max}$)	per-variant: $n_{aE}=0 \rightarrow$ all 0 (masked); $n_{aE}=1 \rightarrow$ <code>inv_softplus(1.0)</code> in slot 0; $n_{aE} \geq 2 \rightarrow$ linear interpolation in inverse-softplus space between <code>inv_softplus(0.25)</code> and <code>inv_softplus(10.0)</code> across the active tiers (lines 904–914)	zeroed when <code>per_neuron=False</code> (line 1039)
log_tau_a_E_gain	(K,)	0.0 (line 916)	always trains

The interpolation in inverse-softplus space differs from the single-cell `SRNNCell` (lines 400–411) which interpolates in **log space** at runtime — this is documented as the multi-tier difference between the two cells and is the math source of `KnownIssues.md` §1 column 2.

4.7 SFA coupling c_E, c_I — line 1216

Code form (E):

```
c_E = exp(log_c_E_gain).view(K, 1, 1)      # (K, 1, 1)
    * softplus(isp_c_E_vec)                  # (K, n_E, max_n_a_E)
```

component	shape	init	gradient mask
isp_c_E_vec	(K, $n_E, n_{aE}^{\max}$)	<code>inv_softplus(0.05)</code> (line 902)	zeroed when <code>per_neuron=False</code> (line 1040)
log_c_E_gain	(K,)	0.0 (line 918)	always trains

Note: c_E is **NOT** scaled by τ_{global} — this is a deliberate departure from the τ family. c is a coupling, not a timescale.

4.8 SFA offset c_E^0, c_I^0 — line 1226

Code form:

```
c_0_E = c_0_E_vec + c_0_E_scalar.view(K, 1, 1)  # (K, n_E, max_n_a_E)
```

component	shape	init (hard-coded)	gradient mask
c_0_E_vec	$(K, n_E, n_{aE}^{\max})$	0 (line 919)	zeroed when per_neuron=False (line 1041)
c_0_E_scalar	$(K,)$	0 (line 920)	always trains

No positivity transform — offsets can go negative.

4.9 STD time constants $\tau_b^{\text{rec}}, \tau_b^{\text{rel}}$ — lines 1154–1172

Code form (E side, recovery; relaxation and I side mirror):

```
tau_b_rec_E = tau_global.unsqueeze(-1)      # (K, 1)
            * exp(log_tau_b_rec_E_gain).view(K, 1) # (K, 1)
            * softplus(isp_tau_b_rec_E_vec)    # (K, n_E)
```

component	shape	init	gradient mask
isp_tau_b_rec_E_vec	(K, n_E)	inv_softplus(1.0) (line 960)	zeroed when per_neuron=False (line 1049)
log_tau_b_rec_E_gain	$(K,)$	0.0 (line 962)	always trains
isp_tau_b_rel_E_vec	(K, n_E)	inv_softplus(0.25) (line 964)	zeroed when per_neuron=False (line 1050)
log_tau_b_rel_E_gain	$(K,)$	0.0 (line 966)	always trains

I-side analogues ($*_I_vec, *_I_gain$) at lines 974–987 with the same hard-coded inits.

4.10 Per-variant b -zero-floor flag — lines 1087–1090

```
std_zero_floor_mask = float(c.std_zero_floor)      # (K, 1, 1) buffer (non-persistent)
```

Per-variant 0/1 gating; non-persistent so old checkpoints load cleanly. Used to rescale b via $b \mapsto (b - b_{\min})/(1 - b_{\min})$ at readout (§ 5.3), with $b_{\min} = \tau^{\text{rel}}/(\tau^{\text{rec}} + \tau^{\text{rel}})$ the steady-state value at $r = 1$.

5. Forward dynamics — equations in code-form variable names

This section is the literal RHS implemented by `_compute_b_full`, `_batched_compute_rhs` and `_batched_step_semi_implicit`. The next section (§ 6) restates the same equations in single-letter LaTeX.

5.1 Effective potential (SFA subtraction) — lines 1457–1472

```
sfa_E_contrib[i] = sum_k c_E_masked[i, k] * a_E[i, k]      # (K, B, n_E)
sfa_I_contrib[j] = sum_k c_I_masked[j, k] * a_I[j, k]      # (K, B, n_I)
```

```
x_eff[:, :n_E] = x[:, :n_E] - sfa_E_contrib
x_eff[:, n_E:] = x[:, n_E:] - sfa_I_contrib
```

where $c_E_masked = c_E * sfa_E_mask$ (and same for I) zeroes inactive SFA tiers.

5.2 Firing rate — line 1475

```
r = piecewise_sigmoid( x_eff - a_0.unsqueeze(1) )          # (K, B, N)
```

`piecewise_sigmoid` (lines 40–75) is a C^1 approximation to a sigmoid: it is 0 for $x < x_1$, then a quadratic rise on $[x_1, x_2]$, then linear on $[x_2, x_3]$ with slope 1, then a quadratic cap on $[x_3, x_4]$, then 1 for $x > x_4$, with $x_{1..4} = c \pm a$, $c \pm 1 \mp a$ where $a = S_a/2$, $c = S_c$, and $k = 0.5/(1 - 2a)$. Defaults: $S_a = 0.9$, $S_c = 0$.

5.3 STD-rescaled synaptic gain b_{full} — lines 1174–1208

For the E side (I mirrors):

```
tau_rec_E    = self._tau_b_rec_E().unsqueeze(1)          # (K, 1, n_E)
tau_rel_E    = self._tau_b_rel_E().unsqueeze(1)
b_min_E      = tau_rel_E / (tau_rec_E + tau_rel_E)        # steady state at r=1
b_E_rescaled = (b_E - b_min_E) / (1 - b_min_E)           # zero-floor map
b_E_used     = b_E * (1 - flag) + b_E_rescaled * flag     # blend by std_zero_floor_mask
b_full_E     = b_E_used * std_E_mask + (1 - std_E_mask)   # 1 when STD inactive
```

The full (K, B, N) vector is $b_{\text{full}} = \text{cat}([b_{\text{full_E}}, b_{\text{full_I}}], -1)$.

5.4 Recurrent drive — lines 1397–1419

```
br = b_full * r                                          # (K, B, N)
Wbr = bmm(W_eff, br_col).squeeze(-1)                   # (K, B, N)
```

(Implementation expands W_{eff} to (KB, N, N) , reshapes br to $(KB, N, 1)$, runs `torch.bmm`, reshapes back.)

5.5 Dendritic ODE — line 1574 (RHS) / line 1488 (semi-implicit)

```
dx_dt = (-x + u + Wbr) / tau_d                          # explicit / RK4 RHS
```

Semi-implicit closed form (Backward-Euler on the linear part):

```
alpha_x = dt / tau_d                                     # (K, 1, N)
x_new   = (x + alpha_x * (u + Wbr)) / (1 + alpha_x)
```

Exponential Euler (line 1663):

```
decay_x = exp(-dt / tau_d)
x_new   = x * decay_x + (1 - decay_x) * (u + Wbr)
```

5.6 SFA ODE — lines 1497, 1581 (E; I mirrors)

```
da_E_dt[i, k] = ( -a_E[i, k] + c_0_E[i, k] + r_E[i] ) / tau_a_E[i, k]
               * sfa_E_mask[k]
```

Semi-implicit:

```
alpha_a_E = dt / tau_a_E                                # (K, n_E, max_n_a_E)
a_E_updated = (a_E + alpha_a_E * (c_0_E + r_E)) / (1 + alpha_a_E)
a_E_new     = a_E * (1 - sfa_E_mask) + a_E_updated * sfa_E_mask
```

(The mask blend keeps unused tiers at their padded value of 0 so that the gradient never flows through inactive tiers.)

5.7 STD ODE — lines 1521, 1598 (E; I mirrors)

```
db_E_dt[i] = (1 - b_E[i]) / tau_b_rec_E[i]
            - r_E[i] * b_E[i] / tau_b_rel_E[i]
db_E_dt    *= std_E_mask
```

Semi-implicit (Backward-Euler on both linear parts; the bilinear $b \cdot r$ term is treated implicitly in b):

```
b_E_updated = (b_E + dt / tau_b_rec_E)
              / (1 + dt * (1 / tau_b_rec_E + r_E / tau_b_rel_E))
b_E_updated = b_E_updated.clamp(0, 1)
b_E_new     = b_E * (1 - std_E_mask) + b_E_updated * std_E_mask
```

The `clamp(0, 1)` enforces the physical range of an available-resource fraction; without it, large explicit-Euler steps can briefly leave the simplex.

5.8 Readout & skip — line 1750 (cell), `sequence_model.py:250` (skip)

The cell emits one of three pre-readout signals according to per-variant `readout_ids`:

```

out_synaptic = b_last * r_last
out_rate     = r_last
out_dendritic = x

```

```

is_synaptic  = (readout_ids == 0).float().reshape(K, 1, 1)
is_rate      = (readout_ids == 1).float().reshape(K, 1, 1)
is_dendritic = (readout_ids == 2).float().reshape(K, 1, 1)

```

```

output = is_synaptic * out_synaptic + is_rate * out_rate + is_dendritic * out_dendritic

```

In `SequenceModel`, this output is multiplied by the optional `output_mask`, projected through a (K, O, N) readout head with a per-variant scalar gain on the weight, then optionally augmented with a per-variant skip residual:

```

W_out = W_out_gain.view(K, 1, 1) * readout_weight
logits = einsum("k...e,koe->k...o", out_t, W_out) + readout_bias
if any_skip:
    logits = logits + skip_flags.view(K, 1, 1) * x_in_t      # autoregressive only

```

`W_out_gain` is a free-real per-variant scalar (init 1.0) that scales the readout weight only — bias is left alone, matching the `W_in_gain` / `W_raw_gain` pattern on the cell. `skip_flags` is the $(K,)$ buffer registered at line 1001; non-skip variants contribute 0. The skip residual is in OUTPUT space (post-readout, post-gain), not state space.

6. The same equations in LaTeX symbols

Drop the K axis (one variant). Let

- $i \in \{1, \dots, n_E\}$ index excitatory neurons (E), and use $\bar{i} \in \{n_E + 1, \dots, N\}$ for inhibitory (I). Where convenient we treat all neurons uniformly with $i \in \{1, \dots, N\}$ and let the side-specific quantities be selected by membership.
- $k \in \{1, \dots, n_a^{(i)}\}$ index SFA tiers, where $n_a^{(i)} = n_{aE}$ if $i \leq n_E$, else n_{aI} .

6.1 Effective inputs

$$x_i^{\text{eff}} = x_i - \sum_{k=1}^{n_a^{(i)}} c_{i,k} a_{i,k}$$

$$r_i = \phi(x_i^{\text{eff}} - a_{0,i}), \quad \phi = \text{piecewise_sigmoid}$$

$$\tilde{b}_i = \begin{cases} \frac{b_i - b_{\min,i}}{1 - b_{\min,i}}, & \text{if } \text{std_zero_floor} \\ b_i, & \text{otherwise} \end{cases}, \quad b_{\min,i} = \frac{\tau_i^{\text{rel}}}{\tau_i^{\text{rec}} + \tau_i^{\text{rel}}}$$

$$u = W^{\text{in,eff}} u_{\text{ext}}, \quad W^{\text{in,eff}} = g^{\text{in}}(W^{\text{in}} \odot M^{\text{in}})$$

6.2 Recurrent weight

$$W_{ij}^{\text{eff}} = g^W \cdot [d \cdot s_j \cdot \text{softplus}(W_{ij}^{\text{raw}}) + (1 - d) W_{ij}^{\text{raw}}] \cdot M_{ij}^{\text{sp}}$$

with $d \in \{0, 1\}$ the per-variant Dale's flag, $s_j \in \{+1, -1\}$ the sign of column j , M^{sp} the sparsity mask, and g^W the trainable scalar `W_raw_gain`.

6.3 ODE system

$$\tau_{d,i} \dot{x}_i = -x_i + u_i + \sum_{j=1}^N W_{ij}^{\text{eff}} \tilde{b}_j r_j \quad (\text{line 1574})$$

$$\tau_{a,(i,k)} \dot{a}_{i,k} = -a_{i,k} + c_{(i,k)}^0 + r_i \quad (\text{lines 1577–1591})$$

$$\dot{b}_i = \frac{1 - b_i}{\tau_i^{\text{rec}}} - \frac{b_i r_i}{\tau_i^{\text{rel}}} \quad (\text{lines 1594–1608})$$

(SFA and STD equations are multiplied by their respective binary masks `sfa*_mask`, `std*_mask`, which gate the entire RHS to 0 for inactive tiers / sides.)

6.4 Time-constant decompositions

For each $\tau \in \{\tau_d, \tau_a^E, \tau_a^I, \tau_b^{\text{rec},E}, \tau_b^{\text{rel},E}, \tau_b^{\text{rec},I}, \tau_b^{\text{rel},I}\}$:

$$\tau_{*,i} = \underbrace{\text{softplus}(\theta^{\tau, \text{global}})}_{\tau_{\text{global}}} \cdot \underbrace{\exp(\theta^{\tau, \text{gain}})}_{g^\tau} \cdot \underbrace{\text{softplus}(\theta_i^{\tau, \text{vec}})}_{\text{per-neuron}}$$

For SFA coupling c (E and I), the formula is the same minus the τ_{global} factor:

$$c_{(i,k)} = \exp(\theta^{c, \text{gain}}) \cdot \text{softplus}(\theta_{(i,k)}^{c, \text{vec}})$$

For all additive parameters $\theta \in \{a_0, c_E^0, c_I^0\}$:

$$\theta_i = \theta_i^{\text{vec}} + \theta^{\text{scalar}}$$

with no positivity transform. Per-neuron splits are enabled by zeroing the gradient on θ^{vec} when `per_neuron=False`, leaving the optimizer to move only θ^{scalar} — i.e. the “per-neuron” term collapses to “shared across neurons.”

6.5 Solvers

For brevity write the dendritic ODE generically as $\tau \dot{x} = -x + F(x, t)$ with $F(x, t) = u + W\tilde{b}r$ taken **explicit** in x inside one substep.

- **Explicit Euler** (line 1612): $x \leftarrow x + \Delta t (-x + F)/\tau$.
- **Semi-implicit / linearly-implicit Euler** (line 1488): treat the $-x/\tau$ term implicitly, $x \leftarrow (x + \alpha F)/(1 + \alpha)$ with $\alpha = \Delta t/\tau$.
- **RK4** (line 1626): four evaluations of `_batched_compute_rhs`, classical weights $\frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$.
- **Exponential Euler** (line 1654): $x \leftarrow x e^{-\Delta t/\tau} + (1 - e^{-\Delta t/\tau}) F$.

The same template is applied analogously to the SFA equation ($-a/\tau$ implicit, r/τ explicit). The STD equation uses the *bilinear-implicit* form $b_{\text{new}} = (b + \Delta t/\tau^{\text{rec}})/(1 + \Delta t \cdot (1/\tau^{\text{rec}} + r/\tau^{\text{rel}}))$ in the semi-implicit solver, and explicit Euler in the exponential solver.

7. Ablation masks — full listing

All masks are buffers (not parameters); they are derived from the per-variant `SRNNConfig` at `__init__` time and are constant across training. Multiplicative masks live in $\{0, 1\}$.

mask	shape	source field	effect
<code>dales_mask</code>	$(K, 1, 1)$	<code>cfg.dales</code>	gates the softplus + sign branch in <code>_effective_W</code> (line 1129)
<code>dales_signs</code>	(K, N)	<code>rmt_export["dales_sign"]</code>	column sign ± 1 ; multiplied into W^{eff} when Dale’s is on
<code>sparsity_masks</code>	(K, N, N)	<code>rmt_export["sparsity_mask"]</code>	structural zeros in W^{eff}
<code>echo_flags</code>	$(K, 1, 1)$	<code>cfg.echo</code>	source for <code>_echo_grad_mask = 1 - echo_flags</code> , applied as a backward hook on <code>W_raw</code> (line 1008)

mask	shape	source field	effect
<code>_echo_grad_mask</code>	$(K, 1, 1)$	derived	zeros gradient on <code>W_raw</code> for echo variants; <code>W_raw_gain</code> is unaffected
<code>skip_flags</code>	$(K,)$	<code>cfg.skip</code>	adds <code>flag * x_in_t</code> to logits in the readout (sequence_model.py:257)
<code>sfa_E_mask</code> , <code>sfa_I_mask</code>	$(K, 1, n_a^{\max})$	<code>cfg.n_a_E</code> , <code>cfg.n_a_I</code>	per-tier 0/1; zeroes inactive SFA tier contributions to x^{eff} and to $\dot{a}$
<code>std_E_mask</code> , <code>std_I_mask</code>	$(K, 1)$	<code>cfg.n_b_E > 0</code> , <code>cfg.n_b_I > 0</code>	gates b_{full} to 1 and $\dot{b}$ to 0 when STD off
<code>std_zero_floor_mask</code>	$(K, 1, 1)$, non-persistent	<code>cfg.std_zero_floor</code>	blends raw b vs. $(b - b_{\min}) / (1 - b_{\min})$ in <code>_compute_b_full</code>
<code>readout_ids</code>	$(K,)$ long	<code>cfg.readout</code>	integer code 0 / 1 / 2 ; selects <code>b·r</code> vs. <code>r</code> vs. <code>x</code> at output
<code>_a_0_vec_mask</code>	$(K, 1)$	<code>cfg.per_neuron</code>	gradient hook on <code>a_0_vec</code> (line 1034)
<code>_isp_tau_d_vec_mask</code>	$(K, 1)$	<code>cfg.per_neuron</code>	gradient hook on <code>isp_tau_d_vec</code> (line 1036)
<code>_isp_tau_a_E_vec_mask</code>	$(K, 1, 1)$	<code>cfg.per_neuron</code>	gradient hook on <code>isp_tau_a_E_vec</code> (line 1039)
<code>_isp_c_E_vec_mask</code>	$(K, 1, 1)$	<code>cfg.per_neuron</code>	gradient hook on <code>isp_c_E_vec</code> (line 1040)
<code>_c_0_E_vec_mask</code>	$(K, 1, 1)$	<code>cfg.per_neuron</code>	gradient hook on <code>c_0_E_vec</code> (line 1041)
<code>_isp_tau_a_I_vec_mask</code> etc.	$(K, 1, 1)$	<code>cfg.per_neuron</code>	I-side analogues, lines 1043–1046
<code>_isp_tau_b_rec_E_vec_mask</code>	$(K, 1)$	<code>cfg.per_neuron</code>	gradient hooks (lines 1049–1050)
<code>_isp_tau_b_rel_E_vec_mask</code>			
<code>_isp_tau_b_rec_I_vec_mask</code>	$(K, 1)$	<code>cfg.per_neuron</code>	gradient hooks (lines 1053–1054)
<code>_isp_tau_b_rel_I_vec_mask</code>			
<code>W_in_mask</code>	$(1, N, 1)$	external arg from <code>factory.build_batched_model</code>	zeros W^{in} rows for non-input neurons (the ~25 % input partition)

7.1 Per-neuron linking (the $\theta = \theta^{\text{vec}} + \theta^{\text{scalar}}$ pattern)

The `per_neuron` flag controls **only** whether the `*_vec` component receives gradient updates. The scalar / gain component **always** trains. As a result:

<code>per_neuron</code>	scalar / gain trains	vec trains	semantics
False	yes	no (gradient masked to 0)	shared per-variant value; vec stays at hard-coded init
True	yes	yes	scalar is shared direction; vec is per-neuron deviation

This is the “linking” architecture documented in `KnownIssues.md` §1. It applies uniformly to every per-neuron parameter: `a_0`, `isp_tau_d`, `isp_tau_a_E/I`, `isp_c_E/I`, `c_0_E/I`, `isp_tau_b_rec_E/I`, `isp_tau_b_rel_E/I` (i.e. 11 parameter families).

8. Initialization summary table

Every initial value used at construction time, with its source. “Hard-coded” means the value is a Python literal in `srnn_cell.py` and is **not** overrideable through `SRNNConfig` or YAML. Overrides can still be made by editing the file or by post-construction

parameter assignment, but not via the standard config path.

parameter	init expression	hard-coded?	line
W_raw	rmt_export["W_init"]	no — comes from RMTMatrix builder in factory	859
W_in	0.1 * randn(K, N, D)	yes — scale 0.1	864
W_raw_gain	ones(K)	yes	877
W_in_gain	ones(K)	yes	878
a_0_vec	full((K, N), 0.35)	yes	885
a_0_scalar	zeros(K)	yes	886
isp_tau_global	inv_softplus(c.tau_global_init)	hideable (cfg.tau_global_init)	889–891
isp_tau_d_vec	inv_softplus(0.1)	yes	896
log_tau_d_gain	zeros(K)	yes	897
isp_tau_a_E_vec	per-tier; n_a_E=1 → inv_softplus(1.0); n_a_E≥2 → linear interp in inverse-softplus space between inv_softplus(0.25) and inv_softplus(10.0)	yes	904–914
log_tau_a_E_gain	zeros(K)	yes	916
isp_c_E_vec	inv_softplus(0.05)	yes	902
log_c_E_gain	zeros(K)	yes	918
c_0_E_vec, c_0_E_scalar	0	yes	919–920
isp_tau_a_I_*, isp_c_I_*, c_0_I_*	I-side mirrors of E above	yes	930–948
isp_tau_b_rec_E_vec	inv_softplus(1.0)	yes	960
log_tau_b_rec_E_gain	zeros(K)	yes	962
isp_tau_b_rel_E_vec	inv_softplus(0.25)	yes	964
log_tau_b_rel_E_gain	zeros(K)	yes	966
isp_tau_b_*_I_*	I-side mirrors	yes	974–982
state a_E, a_I	0	yes (matches MATLAB)	init_state
state b_E, b_I	1	yes	init_state
state x	0.1 · $\mathcal{N}(0, 1)$	yes	init_state

About freeze_params. A recent CLI knob (commit 34600c5) lets the user pin selected parameters at their init values for the whole run. This is implemented at the *training-loop* / *parameter-group* level, not inside the cell — `BatchedSRNNCell` itself is unaware of it and would behave identically with or without the knob.

9. Effective values vs stored values — quick reference

When inspecting a checkpoint by hand, remember:

stored	effective in the ODE	

isp_tau_global	->	softplus(.)
isp_tau*_vec	->	softplus(.) (so 0 -> ~0.693 s baseline)
log_tau*_gain	->	exp(.) (so 0 -> 1.0 multiplier)
isp_c*_vec	->	softplus(.)
log_c*_gain	->	exp(.)
c_0*_vec, c_0*_scalar	->	identity (additive)
a_0_vec, a_0_scalar	->	identity (additive)
W_raw_gain, W_in_gain, W_out_gain	->	identity (multiplicative; can go negative)
W_raw (Dale's on)	->	dales_sign · softplus(.)
W raw (Dale's off)	->	identity

The plot helpers in `scripts/plots/` (e.g. `plot_all_taus.py`, `weight_evolution.png`) and the param tables produced by `scripts/postprocess.py` always report **effective** values to avoid the mental gymnastics above.

10. Inverse transforms (init helpers)

Two helpers in lines 26–37:

```
inv_softplus(x: float) -> log(expm1(x))          # scalar, used at __init__
_softplus_inv(x: Tensor) -> torch.log(torch.expm1(x)) # tensor
```

Both require $x > 0$. Numerically stable for large x (where $\text{softplus}(y) \approx y$), they reduce to $\log(e^x - 1)$.

There is no helper for `exp` because every log-gain is initialised to $\log 1 = 0$ directly.

11. Side-by-side: stored parameter $\leftrightarrow$ effective ODE coefficient

A condensed all-in-one cheatsheet of every quantity that appears on the RHS of the ODE.

ODE symbol	stored params (code)	line	effective expression (code)
x_i	<code>state["x"]</code>	1277	— (state)
$a_{i,k}^E$	<code>state["a_E"]</code>	1249	— (state)
$a_{j,k}^I$	<code>state["a_I"]</code>	1257	— (state)
b_i^E	<code>state["b_E"]</code>	1264	— (state)
b_j^I	<code>state["b_I"]</code>	1271	— (state)
u_i	<code>W_in, W_in_gain,</code> <code>W_in_mask</code>	1423–1447	<code>W_in_gain * (W_in * W_in_mask) @ inputs.T</code>
W_{ij}^{eff}	<code>W_raw, W_raw_gain,</code> <code>dales_signs, dales_mask,</code> <code>sparsity_masks</code>	1117–1131	see § 4.1
$a_{0,i}$	<code>a_0_vec, a_0_scalar</code>	1212	<code>a_0_vec + a_0_scalar.view(K,1)</code>
τ_{global}	<code>isp_tau_global</code>	1135	<code>softplus(isp_tau_global)</code>
$\tau_{d,i}$	<code>log_tau_d_gain,</code> <code>isp_tau_d_vec, plus τ_{global}</code>	1139	<code>tau_global * exp(log_tau_d_gain) * softplus(isp_tau_d_vec)</code>
$\tau_{a,(i,k)}^E$	<code>log_tau_a_E_gain,</code> <code>isp_tau_a_E_vec, plus</code> <code>τ_{global}</code>	1144	<code>tau_global * exp(log_tau_a_E_gain) * softplus(isp_tau_a_E_vec)</code>
$\tau_{a,(j,k)}^I$	<code>log_tau_a_I_gain,</code> <code>isp_tau_a_I_vec, plus</code> <code>τ_{global}</code>	1149	analogous
$c_{(i,k)}^E$	<code>log_c_E_gain, isp_c_E_vec</code>	1216	<code>exp(log_c_E_gain) * softplus(isp_c_E_vec)</code>
$c_{(j,k)}^I$	<code>log_c_I_gain, isp_c_I_vec</code>	1221	analogous
$c_{(i,k)}^{0,E}$	<code>c_0_E_vec, c_0_E_scalar</code>	1226	<code>c_0_E_vec + c_0_E_scalar.view(K,1,1)</code>
$c_{(j,k)}^{0,I}$	<code>c_0_I_vec, c_0_I_scalar</code>	1230	analogous
$\tau_{b,i}^{\text{rec},E}$	<code>log_tau_b_rec_E_gain,</code> <code>isp_tau_b_rec_E_vec, plus</code> <code>τ_{global}</code>	1154	<code>tau_global * exp(.) * softplus(.)</code>
$\tau_{b,i}^{\text{rel},E}$	<code>log_tau_b_rel_E_gain,</code> <code>isp_tau_b_rel_E_vec, plus</code> <code>τ_{global}</code>	1159	analogous
$\tau_{b,j}^{\text{rec},I}$	<code>log_tau_b_rec_I_gain,</code> <code>isp_tau_b_rec_I_vec, plus</code> <code>τ_{global}</code>	1164	analogous

ODE symbol	stored params (code)	line	effective expression (code)
$\tau_{b,j}^{\text{rel},I}$	log_tau_b_rel_I_gain, isp_tau_b_rel_I_vec, plus τ_{global} (derived)	1169	analogous
$b_{\min,i}$		1190, 1200	tau_b_rel / (tau_b_rec + tau_b_rel)
$\tilde{b}_i$	$b_i + \text{std_zero_floor_mask}$ + $\text{std_E_mask}/\text{std_I_mask}$ (no params)	1174–1208	see § 5.3
$\phi(\cdot)$		40–75	piecewise-linear with quadratic edges

12. Inconsistencies worth resolving

Surfaced by writing this doc. Recording them here so the cleanup pass has a checklist:

1. **Mixed positivity transforms inside one τ .** `softplus` for `isp*_vec`, `exp` for `log*_gain`. Picking one would simplify reasoning and gradient analysis. Note `exp` was likely chosen for the gain parameters because it gives true multiplicative identity at $\theta = 0$, while `softplus(0) = log 2  $\approx$  0.693` would offset the baseline.
2. ~~**log*_vec is misnamed.**~~ **Resolved.** Inverse-softplus parameters are now prefixed `isp_*` (e.g. `isp_tau_d_vec`, `isp_tau_a_E_vec`). Genuine log-space scalars keep `log*_gain`. The transform is now a pure prefix lookup.
3. **W_raw_gain, W_in_gain, W_out_gain are unconstrained.** They initialise to +1 but can be driven to any sign by the optimizer. Decide whether to (a) leave free, (b) constrain to $\mathbb{R}_{>0}$ via `softplus`, or (c) parameterise in log space. Same call as for the timescale gains.
4. **c_E skips the τ_{global} factor** but otherwise uses the same gain $\times$ vec template as the time constants. Document or re-evaluate.
5. **Multi-tier SFA init disagrees between SRNNCell and BatchedSRNNCell.** Single-cell interpolates in **log** space at runtime (line 411); batched interpolates in **inverse-softplus** space at init (line 913). See `KnownIssues.md` §1.

13. Symbol cheatsheet

LaTeX	code	meaning	shape (one variant)
x_i	<code>x</code>	dendritic potential	(B, N)
$a_{i,k}$	<code>a_E[i,k], a_I[j,k]</code>	SFA state, tier k	$(B, n_*, n_a^{\max})$
b_i	<code>b_E[i], b_I[j]</code>	available-resource fraction (STD)	(B, n_*)
r_i	<code>r</code>	firing rate	(B, N)
$\tilde{b}_i$	<code>b_full</code>	effective synaptic gain	(B, N)
u_i	<code>u</code>	external drive	(B, N)
W_{ij}^{eff}	<code>W_eff</code>	effective recurrent weight	(N, N)
$W^{\text{in,eff}}$	<code>W_in_eff</code>	effective input weight	(N, D)
$a_{0,i}$	<code>_a_0()</code>	threshold	$(N,)$
τ_d	<code>_tau_d()</code>	dendritic time constant	$(N,)$
$\tau_{a,(i,k)}$	<code>_tau_a_E(), _tau_a_I()</code>	SFA time constant	$(n_*, n_a^{\max})$
$c_{(i,k)}$	<code>_c_E(), _c_I()</code>	SFA coupling	$(n_*, n_a^{\max})$
$c_{(i,k)}^0$	<code>_c_0_E(), _c_0_I()</code>	SFA offset	$(n_*, n_a^{\max})$
$\tau_b^{\text{rec}}, \tau_b^{\text{rel}}$	<code>_tau_b_rec_E/I(),</code> <code>_tau_b_rel_E/I()</code>	STD time constants	$(n_*,)$
$b_{\min}$	(derived in <code>_compute_b_full()</code>)	STD zero-floor	$(n_*,)$
g^W	<code>W_raw_gain</code>	recurrent gain	$()$
g^{in}	<code>W_in_gain</code>	input gain	$()$
g^{out}	<code>W_out_gain</code> (on <code>SequenceModel</code>)	readout-weight gain	$()$
τ_{global}	<code>_tau_global()</code>	global timescale multiplier	$()$

LaTeX	code	meaning	shape (one variant)
d	<code>dales_mask</code>	Dale's flag	()
s_j	<code>dales_signs</code>	column sign ± 1	$(N,)$
M^{sp}	<code>sparsity_masks</code>	sparsity mask	(N, N)
M^{in}	<code>W_in_mask</code>	input-neuron partition mask	$(N, 1)$
ϕ	<code>piecewise_sigmoid</code>	bounded $[0, 1]$ activation	scalar fn
Δt	<code>dt = h / ode_unfolds</code>	solver substep	scalar